



移动智能终端生态联盟——
16KB Page Size 适配指导

编制单位：移动智能终端生态联盟·标准工作组

2025 年 7 月

目录

一、相关定义	3
1.1 页 (Page)：页是内存划分的逻辑单位。	3
1. 作用：	3
2. 特点：	3
1.2. 页面大小 (Page Size)：页面大小是指每个页所包含的字节数。	3
1. 常见值：	3
2. 影响：	3
二、适配建议	3
2.1. 检查应用是否受到影响	3
2.1.1 使用 APK 分析器识别原生库	4
2.1.2 检查共享库的 ELF 段对齐情况	5
2.2. 构建支持 16 KB 设备的应用	6
2.2.1 更新共享库的打包	7
2.2.2 使用 16 KB ELF 对齐方式编译应用	7
2.2.3 修复代码并解决运行时问题	9
2.2.4 检查 SDK 是否支持 16 KB	10
三、如何进行测试	10

一、相关定义

1.1 页（Page）：页是内存划分的逻辑单位。

1. 作用：
 - a. 内存被划分为一个个固定大小的页，便于操作系统进行内存管理。通过页表等数据结构，操作系统可以跟踪每个页的状态，如是否被占用、在物理内存中的位置等。
 - b. 实现虚拟内存机制。程序使用的虚拟地址空间被划分为页，当程序访问某个虚拟地址时，操作系统通过页表将虚拟页映射到物理页，使得程序可以在比实际物理内存更大的地址空间中运行。
2. 特点：
 - a. 页是逻辑上的划分，不一定连续地存储在物理内存中。不同的页可以分散存储在物理内存的不同位置，通过页表进行关联

1.2. 页面大小（Page Size）：页面大小是指每个页所包含的字节数。

1. 常见值：
 - a. 常见的页面大小有 4KB（4096 字节）、16KB 等。不同的操作系统和硬件平台可能会采用不同的页面大小。
 - b. 页面大小通常是 2 的整数次幂，这是因为在计算机体系结构中，以 2 的幂次方进行地址计算比较方便，可以通过位移操作快速确定页号和页内偏移量。
2. 影响：
 - a. 对内存管理效率的影响：页面大小的选择会影响内存管理的效率。较小的页面大小可以提高内存分配的灵活性，减少内部碎片，但会增加页表的大小和管理开销；较大的页面大小可以减少页表的大小和内存访问的次数，但可能导致较大的外部碎片。
 - b. 对程序性能的影响：页面大小也会影响程序的性能。如果程序的工作集（程序在一段时间内频繁访问的内存区域）能够较好地适应页面大小，那么可以提高内存访问的效率；如果工作集与页面大小不匹配，可能会导致频繁的页面换入换出，降低程序性能。

二、适配建议

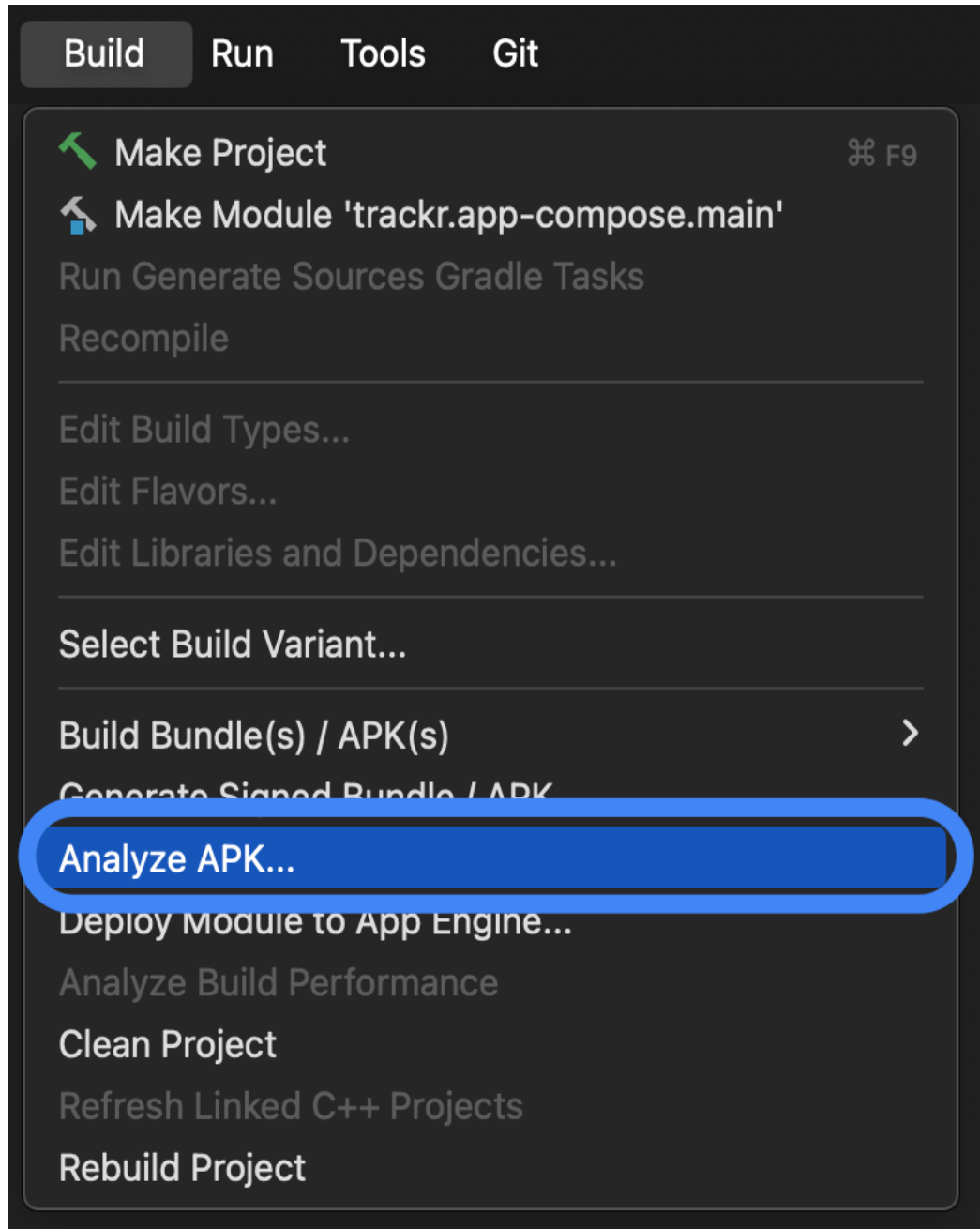
Android 过去仅支持 4 KB 内存页面大小，优化了系统内存性能，以针对 Android 设备通常具备的功能。从 Android 15 开始，AOSP 支持 配置为使用 16 KB（16 KB）页面大小的设备（设备）。如果您的应用使用任何 NDK 库，直接使用或者通过 SDK 间接创建，那么，您需要重新构建自己的应用以支持未来 16KB 设备。

2.1. 检查应用是否受到影响

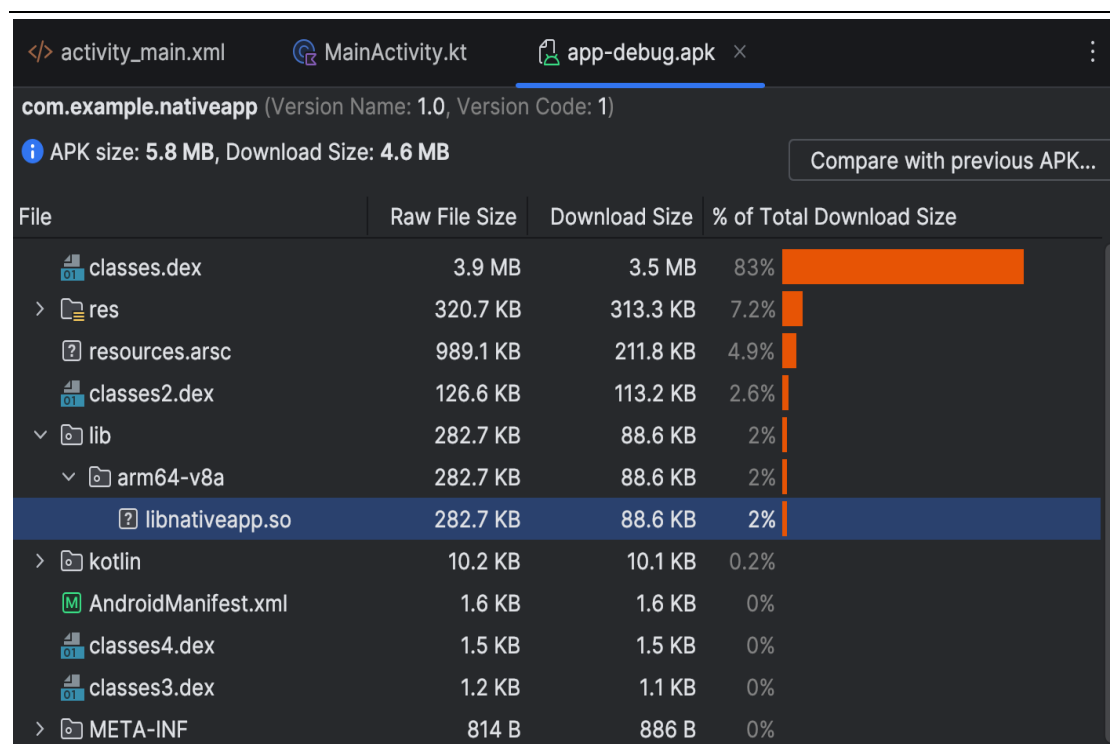
2.1.1 使用 APK 分析器识别原生库

[APK 分析器](#)是一款可用于对所构建的 APK 进行各方面评估的工具。如需检查您的应用是否使用了原生代码（无论其是否兼容 16 KB），请执行以下操作：

1. 打开 *Android Studio*，然后依次点击 **File > Open**，然后选择任意项目。
2. 在菜单栏中，依次点击 **Build > Analyze APK...**



3. 选择要分析的 APK。
4. 查看 `lib` 文件夹，其中会托管 `.so` 文件对象（如有）。如果存在任何 `.so` 对象文件，则表明您的应用使用了原生代码。如果没有 `.so` 文件或没有 `lib` 文件夹，则意味着应用没有使用原生代码，就无需进行适配。



File	Raw File Size	Download Size	% of Total Download Size
classes.dex	3.9 MB	3.5 MB	83%
res	320.7 KB	313.3 KB	7.2%
resources.arsc	989.1 KB	211.8 KB	4.9%
classes2.dex	126.6 KB	113.2 KB	2.6%
lib	282.7 KB	88.6 KB	2%
arm64-v8a	282.7 KB	88.6 KB	2%
libnativeapp.so	282.7 KB	88.6 KB	2%
kotlin	10.2 KB	10.1 KB	0.2%
AndroidManifest.xml	1.6 KB	1.6 KB	0%
classes4.dex	1.5 KB	1.5 KB	0%
classes3.dex	1.2 KB	1.1 KB	0%
META-INF	814 B	886 B	0%

若应用使用了原生代码，则需进行第二步检查共享库（.so 文件）的 ELF 段对齐情况

2.1.2 检查共享库的 ELF 段对齐情况

对于任何共享库，开发者需要验证共享库的 ELF 段是否使用 16 KB ELF 对齐方式正确对齐。如果您是在 Linux 或 macOS 上进行开发，则可以用 `check_elf_alignment.sh` 脚本。

使用 `check_elf_alignment.sh` 脚本（Linux 或 macOS）

如需使用 `check_elf_alignment.sh` 脚本检查 ELF 段的对齐情况，按以下步骤操作：

1. 将 `check_elf_alignment.sh` 脚本保存到文件中。（脚本链接：
https://cs.android.com/android/platform/superproject/main/+main:system/extras/tools/check_elf_alignment.sh?hl=zh-cn）
2. 对应用的 APK 文件运行脚本：

```
check_elf_alignment.sh APK_NAME.apk
```

该脚本会针对所有 `arm64-v8a` 共享库输出 `ALIGNED` 或 `UNALIGNED`。

3. 如果任何 `arm64-v8a` 或 `x86_64` 共享库为 `UNALIGNED`，则需要适配 16k，重新编译应用并按照本部分中的步骤重新测试。

除了使用脚本检查外还可以直接使用命令行工具进行检查

直接使用命令行工具

如需直接使用命令行工具检查 ELF 段的对齐情况，请按以下步骤操作：

1. 确保使用 Android Studio 中的 SDK 管理器或 `sdkmanager` 命令行工具安装了 Android SDK Build-Tools 35.0.0 或更高版本和 Android NDK。
2. 解压缩应用的 APK 文件：

3. 将 APK 文件提取到临时目录中，检查 `lib` 目录中是否有共享对象（.so）文件。如有则对每个共享对象文件运行以下命令：

```
//Linux 或 macOS
SDK_ROOT_LOCATION/Android/sdk/ndk/NDK_VERSION/toolchains/llvm/prebuilt/darwin-x86_64/bin/llvm-objdump -p SHARED_OBJECT_FILE.so | grep LOAD

//windows
SDK_ROOT_LOCATION\Android\sdk\ndk\NDK_VERSION\toolchains\llvm\prebuilt\windows-x86_64\bin\llvm-objdump.exe -p SHARED_OBJECT_FILE.so | select-string -pattern "LOAD"
```

其中 `SDK_ROOT_LOCATION` 是安装 Android SDK 的目录的路径，`SHARED_OBJECT_FILE` 是要检查的共享对象文件的名称，`NDK_VERSION` 是安装的 Android NDK 的版本（例如 28.0.12433566）。检查的每个文件，输出将如下所示：

```
LOAD off      0x0000000000000000 vaddr 0x0000000000000000 paddr 0x0000000000000000
LOAD off      0x00000000000042a90 vaddr 0x00000000000043a90 paddr 0x0000000000000004
LOAD off      0x00000000000046230 vaddr 0x00000000000048230 paddr 0x0000000000000004
```

1. 检查输出行，确保加载分段的值不小于 `2**14`。如果任何加载分段的值为 `2**13`、`2**12` 或更低值，则需要适配 16k，重新编译应用并按照本部分中的步骤重新测试。
2. 接下来，对应用的 APK 文件运行 `zipalign` 命令行工具：

```
//Linux 或 macOS
SDK_ROOT_LOCATION/Android/sdk/build-tools/35.0.0/zipalign -v -c -P 16 4 APK_NAME.apk

//windows
SDK_ROOT_LOCATION\Android\sdk\build-tools\35.0.0\zipalign.exe -v -c -P 16 4 APK_NAME.apk
```

其中 `SDK_ROOT_LOCATION` 是安装 Android SDK 的目录的路径，`APK_NAME` 是应用的 APK 文件的名称。如果所有共享库都正确对齐，输出的最后一行将显示“验证成功”。如果验证失败，则需要重新对某些共享库进行对齐，则需要进一步检查适配，重新编译应用并按照本部分中的步骤重新测试。

2.2. 构建支持 16 KB 设备的应用

2.2.1 更新共享库的打包

建议应用升级到 AGP 8.5.1 或更高版本，并使用未压缩的共享库。16 KB 设备要求随应用一起提供的未压缩共享库在 16 KB ZIP 对齐边界上对齐。为此，开发者需要升级到 Android Gradle 插件 (AGP) 8.5.1 或更高版本。

如果无法将 AGP 升级到 8.5.1 或更高版本，则可以改用压缩共享库。更新 Gradle 配置，以便 Gradle 在打包应用时压缩共享库，从而避免因共享库未对齐而导致应用安装问题。在 `build.gradle` 文件中，添加以下选项：

```
//Groovy
android {
    ...
    packagingOptions {
        jniLibs {
            useLegacyPackaging true
        }
    }
}

//Kotlin
android {
    ...
    packagingOptions {
        jniLibs {
            useLegacyPackaging = true
        }
    }
}
```

2.2.2 使用 16 KB ELF 对齐方式编译应用

16 KB 设备要求共享库的 ELF 段使用 16 KB ELF 对齐方式正确对齐，才能运行应用。

对于游戏开发者，如果游戏是在 Unity 游戏引擎上运行的，请参阅 [Unity 指南](#)。如果游戏在 Unreal 游戏引擎上运行，请参阅 [Unreal 指南](#)。

注意：如果应用未将原生库提取到文件系统（`extractNativeLibs` 设置为 `false`），那么在使用 16 KB ELF 对齐方式进行编译后，可能会发现应用的二进制文件大小略有增加。[Android 15](#) 中对软件包管理器的优化可抵消因增加而产生的运行时开销。

如需使用 16 KB ELF 对齐方式编译应用，需根据使用的 Android NDK 版本，完成以下某一部分中的步骤。

重要提示： 如果应用使用了任何预构建的共享库，您还必须以相同的方式重新编译这些库，并将对齐到 **16 KB** 的库重新导入到您的应用中。

Android NDK r28 及更高版本

NDK 版本 r28 及更高版本默认编译为 16 KB 对齐。

Android NDK r27

如需支持使用 Android NDK 版本 r27 及更高版本编译 16 KB 对齐的共享库，需要更新 `ndk-build`、`build.gradle`、`build.gradle.kts` 或链接器标志，如下所示：

```
//ndk-build
在 Application.mk 中：
APP_SUPPORT_FLEXIBLE_PAGE_SIZES := true

//Groovy
在 build.gradle 文件中，设置实参 -DANDROID_SUPPORT_FLEXIBLE_PAGE_SIZES=ON：
android {
    ...
    defaultConfig {
        ...
        // This block is different from the one you use to link Gradle
        // to your CMake or ndk-build script.
        externalNativeBuild {
            // For ndk-build, instead use the ndkBuild block.
            cmake {
                // Passes optional arguments to CMake.
                arguments "-DANDROID_SUPPORT_FLEXIBLE_PAGE_SIZES=ON"
            }
        }
    }
}

//kotlin
在 build.gradle.kts 文件中，设置实参 -DANDROID_SUPPORT_FLEXIBLE_PAGE_SIZES=ON
android {
    ...
    defaultConfig {
        ...
        // This block is different from the one you use to link Gradle
        // to your CMake or ndk-build script.
        externalNativeBuild {
```

```
// For ndk-build, instead use the ndkBuild block.
cmake {
    // Passes optional arguments to CMake.
    arguments += ListOf("-DANDROID_SUPPORT_FLEXIBLE_PAGE_SIZES=ON")
}
}
}
```

Android NDK r26 及更低版本

如需支持使用 Android NDK 版本 r26 或更低版本编译 16 KB 对齐的共享库，则需要更新 ndk-build 或 cmake 配置，如下所示：

```
//ndk-build
在 Application.mk 中：
LOCAL_LDFLAGS += "-Wl,-z,max-page-size=16384"

//CMake
更新 CMakeLists.txt 以启用 16 KB ELF 对齐：
```

2.2.3 修复代码并解决运行时问题

即使应用已对齐到 16 KB，如果代码中的某些位置假定设备使用了特定的页面大小，应用也可能会遇到错误。为避免这种情况，需完成以下步骤：

移除引用 `PAGE_SIZE` 常量或假定设备页面大小为 4 KB (4096) 的代码逻辑中的实例的任何硬编码依赖项。

1. 改用 `getpagesize()` 或 `sysconf(_SC_PAGESIZE)`。
2. 查找 `mmap()` 和其他需要页面对齐参数的 API 的用法，并在必要时替换为替代方案。在某些情况下，如果应用使用 `PAGE_SIZE` 作为不与底层页面大小相关联的便捷值，则在 16 KB 模式下使用时，这不会导致应用崩溃。不过，如果将此值通过 `mmap` 而非 `MAP_FIXED` 传递给内核，内核仍会使用整个页面，这会浪费一些内存。因此，在 NDK r27 及更高版本中启用 16 KB 模式时，`PAGE_SIZE` 未定义。

如果应用以这种方式使用 `PAGE_SIZE`，并且从不直接将此值传递给内核，请创建一个具有新名称的新变量，而不是使用 `PAGE_SIZE`，以反映该变量用于其他用途，而不是反映实际的内存页面。

2.2.4 检查 SDK 是否支持 16 KB

许多 SDK 都与 16 KB 页面大小兼容，尤其是开发者自行构建 SDK 或获取近期的预构建 SDK 时。不过，由于某些 SDK 预构建或 SDK 版本不支持 16 KB，因此应查看每个 SDK 提供商的网站，以确定应与 16 KB 搭配使用哪个版本。

三、如何进行测试

构建支持 16 KB 设备的应用后，开发者需要在 16 KB 环境中测试应用，以了解应用是否出现任何回归问题。为此，请按以下步骤操作：

1. 设置 Android 15 SDK。
2. 设置以下任一测试环境：
 - 使用基于 16 KB 的 Android 15 系统映像设置 Android 模拟器
 - 使用开发者选项在设备上启用 16 KB 模式
 - 在 ARM64 上使用页面大小为 16KB 的 Cuttlefish
 - 在 x86-64 上模拟页面大小为 16KB 的 Cuttlefish
3. 启动测试设备，然后运行以下命令，验证设备是否使用的是 16 KB 环境：该命令应返回 16384。

```
adb shell getconf PAGE_SIZE
```

4. 运行以下 `zipalign` 命令，验证您的应用是否按 16 KB 对齐，其中 `APK_NAME` 是应用的 APK 文件的名称：

```
zipalign -c -P 16 -v 4 APK_NAME.apk
```

5. 全面测试整个应用，重点关注更改引用特定页面大小的代码实例可能会影响的所有方面。